Monotone–Light Factorization of Compact Hausdorff Maps

Arthur Freitas Ramos
David Barros Hulak
Ruy Jose Guerra Barretto de Queiroz

October 11, 2026

Abstract

Every continuous map from a compact Hausdorff space to a Hausdorff space factors through a compact Hausdorff space as a surjective monotone map followed by a light map. The middle space is the quotient by connected components of point fibres. We prove the factorization and its uniqueness up to a homeomorphism commuting with the factor maps.

Contents

1 Introduction **1**

2 Proof and formal representation **2**

3 Monotone–light factorization **3**
3.1 Lightness and the factorization 8
3.2 Uniqueness 10

1 Introduction

For a continuous map $f : X \to Y$, a monotone map has connected point fibres, and a light map has totally disconnected point fibres. These notions isolate two different behaviours: the collapse of connected subsets, and maps whose fibres have no nontrivial connected subset. The classical monotone–light factorization separates these behaviours. Eilenberg's original result treats compact metric spaces [2]; the compact Hausdorff formulation and its categorical interpretation are discussed by Carboni, Janelidze, Kelly and Paré [1], Sections 7.3–7.5. Michael [4], Section 2, pages 5–6, gives a direct account of fibre components, lightness and Hausdorff separation.

Our statement assumes that X is compact Hausdorff and Y is Hausdorff. The map f need not be surjective, and Y need not be compact. The empty space is allowed. The result constructs a middle space M, a continuous surjective quotient $q : X \to M$, and a continuous map $l : M \to Y$, such that $f = l \circ q$ on the carrier of X. The fibres of q are precisely the connected components of the fibres of f. Consequently q is monotone and l is light.

This is a formalization of a classical theorem, with no claim of new mathematical content. A targeted search of HOL-Analysis and relevant AFP topology entries found no existing formalization of this factorization. The general quotient topology is imported from the AFP entry *Topological Groups* [3]; the development uses HOL-Analysis's existing connected components, quasicomponents, monotone maps, separation theorems and quotient lifting theorem.

Codex assisted with literature searches, proof planning, Isabelle proof engineering, documentation and validation. An independent AI mathematical review checked the argument and the generality of its statements. This disclosure does not assign individual duties to the established human author team or claim human review or endorsement.

2 Proof and formal representation

The carrier of the middle space is the set of fibre components. The projection sends x to the component of x in $f^{-1}\{f(x)\}$, with the quotient topology. Equal projection values characterize membership in the same fibre component. Isabelle represents spaces as topology values with an explicit carrier, so factor identities and uniqueness are asserted on carriers; total functions may have irrelevant values outside them.

The central point is that q is closed. Given a closed $A \subseteq X$ and a point x whose fibre component avoids A, work inside the compact Hausdorff fibre $F = f^{-1}\{f(x)\}$. Components and quasicomponents coincide there. A clopen subset P of F contains the component of x and avoids $A \cap F$. Normality of X gives disjoint open neighbourhoods U, V of P and $A \cup (F \setminus P)$ respectively. Since f is closed, an open neighbourhood N of $f(x)$ has $f^{-1}N \subseteq U \cup V$. A connected fibre component through $U \cap f^{-1}N$ cannot meet V, so this open set is saturated and avoids A. The saturation of every closed A is therefore closed. The quotient criterion proves that q is closed.

Compactness of M follows from the continuous surjection q. A closed continuous image of a normal Hausdorff space is Hausdorff. The quotient lifting theorem now induces l. For a connected subset C of an l-fibre, the closed monotone map q has connected preimage $q^{-1}C$. That preimage is contained in an f-fibre and hence in one component. Surjectivity of q forces C to have at most one point, which proves lightness.

For any alternative continuous monotone–light factorization through a

Hausdorff middle space, with a surjective first map, its first-map fibres equal the fibre components of f: monotonicity gives one inclusion, and lightness of the image of a connected component gives the other. Quotient lifting produces the comparison map. It is a continuous bijection from compact M to a Hausdorff space, hence a homeomorphism. Surjectivity of q makes that comparison unique on the carrier.

All new definitions, statements and complete proofs follow in the formal development.

3 Monotone–light factorization

theory *Monotone-Light*
imports *Topological-Groups.Topological-Group*
HOL−Analysis.Abstract-Topological-Spaces HOL−Analysis.Urysohn
begin

definition *fibre-component* :: *'a topology* ⇒ (*'a* ⇒ *'b*) ⇒ *'a* ⇒ *'a set* **where**
fibre-component X f x =
connected-component-of-set (*subtopology X* {*z* ∈ *topspace X*. *f z* = *f x*}) *x*

definition *component-quotient* **where**
component-quotient X f = *quot-topology X* (*fibre-component X f*)

definition *light-map* **where**
light-map X Y f ⟷ *f* ' *topspace X* ⊆ *topspace Y* ∧
(∀ *y C*. *connectedin X C* ⟶ *C* ⊆ {*x* ∈ *topspace X*. *f x* = *y*}
⟶ (∀ *a*∈*C*. ∀ *b*∈*C*. *a* = *b*))

lemma *fibre-component-subset*:
fibre-component X f x ⊆ {*z* ∈ *topspace X*. *f z* = *f x*}
using *connected-component-of-subset-topspace*
unfolding *fibre-component-def* **by** *fastforce*

lemma *fibre-component-self*:
x ∈ *topspace X* ⟹ *x* ∈ *fibre-component X f x*
unfolding *fibre-component-def* **by** (*simp add*: *connected-component-of-refl*)

lemma *fibre-component-connected*:
connectedin X (*fibre-component X f x*)
unfolding *fibre-component-def*
using *connectedin-connected-component-of*[*of*
subtopology X {*z* ∈ *topspace X*. *f z* = *f x*} *x*]
by (*simp add*: *connectedin-subtopology*)

lemma *fibre-component-eq*:
assumes *z* ∈ *fibre-component X f x*
shows *fibre-component X f z* = *fibre-component X f x*
proof −

have *fx*: $f\ z = f\ x$
using *assms fibre-component-subset*[*of X f x*] **by** *auto*
have *connected-component-of*
(*subtopology X* $\{z \in$ *topspace X*. $f\ z = f\ x\}$) *x z*
using *assms* **unfolding** *fibre-component-def* **by** *simp*
then have *connected-component-of*
(*subtopology X* $\{z \in$ *topspace X*. $f\ z = f\ x\}$) *x* =
connected-component-of (*subtopology X* $\{z \in$ *topspace X*. $f\ z = f\ x\}$) *z*
using *connected-component-of-equiv*[*of*
subtopology X $\{z \in$ *topspace X*. $f\ z = f\ x\}$ *x z*] **by** *blast*
then show *?thesis*
unfolding *fibre-component-def* **using** *fx* **by** *simp*
qed

lemma *fibre-component-fibre*:
assumes $x \in$ *topspace X*
shows $\{z \in$ *topspace X*. *fibre-component X f z* = *fibre-component X f x*$\}$
= *fibre-component X f x*
using *assms fibre-component-eq*[*of - X f x*]
fibre-component-self[*of - X f*] *fibre-component-subset*[*of X f x*] **by** *blast*

lemma *component-quotient-map*:
quotient-map X (*component-quotient X f*) (*fibre-component X f*)
by (*simp add*: *component-quotient-def projection-quotient-map*)

lemma *component-quotient-topspace*:
topspace (*component-quotient X f*) = *fibre-component X f* ‘ *topspace X*
by (*simp add*: *component-quotient-def*)

lemma *component-quotient-monotone*:
monotone-map X (*component-quotient X f*) (*fibre-component X f*)
proof (*unfold monotone-map-def*, *intro conjI ballI*)
show *fibre-component X f* ‘ *topspace X* $\subseteq$ *topspace* (*component-quotient X f*)
by (*simp add*: *component-quotient-topspace*)
next
fix *C* **assume** *C* $\in$ *topspace* (*component-quotient X f*)
then obtain *x* **where** *x*: $x \in$ *topspace X C* = *fibre-component X f x*
by (*auto simp*: *component-quotient-topspace*)
show *connectedin X* $\{z \in$ *topspace X*. *fibre-component X f z* = *C*$\}$
using *fibre-component-fibre*[*OF x*(*1*), *of f*]
by (*simp add*: *x*(*2*) *fibre-component-connected*)
qed

lemma *fibre-component-closed*:
assumes *continuous-map X Y f Hausdorff-space Y*
shows *closedin X* (*fibre-component X f x*)
proof −
have *closedin X* {*z* ∈ *topspace X*. *f z* = *f x*}
proof (*cases f x* ∈ *topspace Y*)
case *True*
have *closedin Y* {*f x*}
by (*rule closedin-Hausdorff-singleton*[*OF assms*(*2*) *True*])
then show *?thesis*
using *closedin-continuous-map-preimage*[*OF assms*(*1*) ‹*closedin Y* {*f x*}›]
by *simp*
next
case *False*
have {*z* ∈ *topspace X*. *f z* = *f x*} = {}
using *continuous-map-image-subset-topspace*[*OF assms*(*1*)] *False* **by** *auto*
then show *?thesis* **by** (*metis closedin-empty*)
qed
then show *?thesis*
unfolding *fibre-component-def*
by (*meson closedin-connected-component-of closedin-trans-full*)
qed

The main separation argument produces an open saturated neighbourhood. Compact Hausdorff separation is central to this argument.

lemma *fibre-component-avoiding-neighbourhood*:
assumes *X*: *compact-space X Hausdorff-space X*
and *f*: *continuous-map X Y f* **and** *Y*: *Hausdorff-space Y*
and *A*: *closedin X A* **and** *x*: *x* ∈ *topspace X*
and *dis*: *disjnt* (*fibre-component X f x*) *A*
obtains *W* **where** *openin X W x* ∈ *W disjnt W A*
⋀*z*. *z* ∈ *W* ⟹ *fibre-component X f z* ⊆ *W*
proof −
let *?F* = {*z* ∈ *topspace X*. *f z* = *f x*}
let *?T* = *subtopology X ?F*
have *fx*: *f x* ∈ *topspace Y*
using *f x continuous-map-image-subset-topspace* **by** *blast*
have *F*: *closedin X ?F*
using *closedin-continuous-map-preimage*[*OF f closedin-Hausdorff-singleton*[*OF Y fx*]]
by *simp*
have *T*: *compact-space ?T Hausdorff-space ?T*
using *X F closedin-compact-space compact-space-subtopology*
Hausdorff-space-subtopology **by** *blast+*
have *qC*: *fibre-component X f x* ∈ *quasi-components-of ?T*
unfolding *fibre-component-def*
using *quasi-eq-connected-components-of*[*of ?T*] *T*
x **by** (*simp add*: *connected-component-in-connected-components-of*)
have *KA*: *compactin ?T* (*?F* ∩ *A*)

```
    using T A closedin-subtopology-Int-closed closedin-compact-space by blast
  have separated-between ?T (fibre-component X f x) (?F ∩ A)
    using separated-between-quasi-component-compact[OF qC KA] dis
    by (auto simp: disjnt-iff)
  then obtain S where S: closedin ?T S openin ?T S
    fibre-component X f x ⊆ S ?F ∩ A ⊆ topspace ?T − S
    by (auto simp: separated-between)
  have FS: S ⊆ ?F closedin X S closedin X (?F − S)
  proof −
    show S ⊆ ?F
      using closedin-subset[OF S(1)] by auto
    show closedin X S
      by (rule closedin-trans-full[OF S(1) F])
    have TF: topspace ?T = ?F by auto
    have closedin ?T (topspace ?T − S)
      using openin-closedin-eq[of ?T S] S(2) by blast
    then have closedin ?T (?F − S) using TF by simp
    then show closedin X (?F − S)
      by (rule closedin-trans-full[OF - F])
  qed
  have normal-space X
    using X compact-Hausdorff-or-regular-imp-normal-space by blast
  moreover have closedin X ((?F − S) ∪ A) disjnt S ((?F − S) ∪ A)
    using FS S A by (auto simp: disjnt-iff)
  ultimately obtain U V where UV: openin X U openin X V
    S ⊆ U (?F − S) ∪ A ⊆ V disjnt U V
    using FS(2) unfolding normal-space-def by meson
  have ?F ⊆ U ∪ V
    using UV by blast
  then obtain N where N: openin Y N f x ∈ N
    {z ∈ topspace X. f z ∈ N} ⊆ U ∪ V
    using closed-map-fibre-neighbourhood[of X Y f]
      continuous-imp-closed-map[OF f X(1) Y] UV fx by blast
  let ?W = U ∩ {z ∈ topspace X. f z ∈ N}
  have W: openin X ?W x ∈ ?W disjnt ?W A
    using UV N f x S fibre-component-self[OF x, of f]
      openin-continuous-map-preimage[OF f N(1)] by (auto simp: disjnt-iff)
  have sat: fibre-component X f z ⊆ ?W if z ∈ ?W for z
  proof −
    have Nsub: fibre-component X f z ⊆ {t ∈ topspace X. f t ∈ N}
      using fibre-component-subset[of X f z] that by auto
    have sub: fibre-component X f z ⊆ U ∪ V
      by (rule subset-trans[OF Nsub N(3)])
    have fibre-component X f z ⊆ U ∨ fibre-component X f z ⊆ V
     using connectedinD[OF fibre-component-connected[of X f z] UV(1,2) sub] UV
sub
      by (auto simp: disjnt-def)
    then have fibre-component X f z ⊆ U
      using that fibre-component-self[of z X f] UV
```

```
    by (auto simp: disjnt-iff)
  then show ?thesis
    using fibre-component-subset[of X f z] that by auto
 qed
 show thesis
  using that W sat by blast
qed

theorem component-quotient-closed:
 assumes compact-space X Hausdorff-space X
   continuous-map X Y f Hausdorff-space Y
 shows closed-map X (component-quotient X f) (fibre-component X f)
proof (unfold closed-map-def, intro allI impI)
 fix A assume A: closedin X A
 let ?q = fibre-component X f
 let ?Q = component-quotient X f
 let ?B = {x ∈ topspace X. ?q x ∈ ?q ‘ A}
 have openin X (topspace X − ?B)
 proof (rule openin-subopen[THEN iffD2], intro ballI)
  fix x assume x: x ∈ topspace X − ?B
  have disjnt (?q x) A
  proof (unfold disjnt-iff, intro allI notI)
   fix z assume z ∈ ?q x ∧ z ∈ A
   then have z: z ∈ ?q x z ∈ A by auto
   have ?q z = ?q x by (rule fibre-component-eq[OF z(1)])
   then have ?q x ∈ ?q ‘ A using z(2) by auto
   then show False using x by auto
  qed
  then obtain W where W: openin X W x ∈ W disjnt W A
   and sat: ⋀z. z ∈ W ⟹ ?q z ⊆ W
   using fibre-component-avoiding-neighbourhood[OF assms A] x by blast
  have W ⊆ topspace X − ?B
  proof (rule subsetI)
   fix z assume z: z ∈ W
   have zX: z ∈ topspace X using openin-subset[OF W(1)] z by blast
   have ?q z ∉ ?q ‘ A
   proof
    assume ?q z ∈ ?q ‘ A
    then obtain a where a: a ∈ A ?q a = ?q z by blast
    have a ∈ topspace X using a(1) closedin-subset[OF A] by blast
    then have a ∈ ?q a by (rule fibre-component-self)
    then have a ∈ W using a(2) sat[OF z] by blast
    then show False using a(1) W(3) by (auto simp: disjnt-iff)
   qed
   then show z ∈ topspace X − ?B using zX by auto
  qed
  then show ∃ T. openin X T ∧ x ∈ T ∧ T ⊆ topspace X − ?B
   using W by blast
 qed
```

then have *closedin X ?B*
unfolding *closedin-def* **by** *auto*
have *imsub*: *?q ‘ A ⊆ topspace ?Q*
using *closedin-subset*[*OF A*] **by** (*auto simp*: *component-quotient-topspace*)
have *closedin X ?B ⟷ closedin ?Q* (*?q ‘ A*)
using *component-quotient-map*[*of X f*] *imsub*
unfolding *quotient-map-closedin* **by** *blast*
then show *closedin ?Q* (*?q ‘ A*) **using** ‹*closedin X ?B*› **by** *blast*
qed

theorem *component-quotient-compact-Hausdorff*:
assumes *compact-space X Hausdorff-space X*
continuous-map X Y f Hausdorff-space Y
shows *compact-space* (*component-quotient X f*)
Hausdorff-space (*component-quotient X f*)
proof −
have *q*: *continuous-map X* (*component-quotient X f*) (*fibre-component X f*)
by (*rule quotient-imp-continuous-map*[*OF component-quotient-map*])
have *compactin X* (*topspace X*)
using *assms*(*1*) **by** (*simp add*: *compact-space-def*)
then have *compactin* (*component-quotient X f*) (*fibre-component X f ‘ topspace X*)
by (*rule image-compactin*[*OF - q*])
then show *compact-space* (*component-quotient X f*)
by (*simp add*: *compact-space-def component-quotient-topspace*)
have *normal*: *normal-space X*
using *assms compact-Hausdorff-or-regular-imp-normal-space* **by** *blast*
have *normal-space* (*component-quotient X f*) ∧ *Hausdorff-space* (*component-quotient X f*)
by (*rule normal-Hausdorff-space-closed-continuous-map-image*[*OF normal assms*(*2*) *component-quotient-closed*[*OF assms*] *q*]) (*simp add*: *component-quotient-topspace*)
then show *Hausdorff-space* (*component-quotient X f*) **by** *blast*
qed

lemmas *component-quotient-compact* = *component-quotient-compact-Hausdorff*(*1*)
lemmas *component-quotient-Hausdorff* = *component-quotient-compact-Hausdorff*(*2*)

3.1 Lightness and the factorization

lemma *component-quotient-light*:
assumes *X*: *compact-space X Hausdorff-space X*
and *f*: *continuous-map X Y f* **and** *Y*: *Hausdorff-space Y*
and *l*: *continuous-map* (*component-quotient X f*) *Y l*
and *eq*: ⋀*x*. *x ∈ topspace X ⟹ l* (*fibre-component X f x*) = *f x*
shows *light-map* (*component-quotient X f*) *Y l*
proof −
let *?Q* = *component-quotient X f*
let *?q* = *fibre-component X f*
have *q*: *continuous-map X ?Q ?q closed-map X ?Q ?q*

```
    using quotient-imp-continuous-map[OF component-quotient-map[of X f]]
      component-quotient-closed[OF X f Y] by blast+
  have pre: connectedin X {x ∈ topspace X. ?q x ∈ C}
    if connectedin ?Q C for C
   using monotone-closed-map[OF q component-quotient-topspace[of X f, symmet-
ric]]
      component-quotient-monotone[of X f] that by blast
  show ?thesis
    unfolding light-map-def
  proof (intro conjI allI impI ballI)
    show l ‘ topspace ?Q ⊆ topspace Y
      by (rule continuous-map-image-subset-topspace[OF l])
  next
    fix y C a b
    assume C: connectedin ?Q C C ⊆ {A ∈ topspace ?Q. l A = y}
      and ab: a ∈ C b ∈ C
    obtain x where xx: x ∈ topspace X ?q x = a
      using C(2) ab(1) by (auto simp: component-quotient-topspace)
    obtain z where zz: z ∈ topspace X ?q z = b
      using C(2) ab(2) by (auto simp: component-quotient-topspace)
    have xz: x ∈ topspace X z ∈ topspace X ?q x = a ?q z = b
      using xx zz by auto
    let ?D = {t ∈ topspace X. ?q t ∈ C}
    have fx: f x = y using eq[OF xx(1)] xx(2) ab(1) C(2) by auto
    have fc: f t = y if t ∈ ?D for t
      using eq[of t] that C(2) by auto
    have csub: ?D ⊆ {t ∈ topspace X. f t = f x}
    proof (rule subsetI)
      fix t assume t: t ∈ ?D
      have f t = y by (rule fc[OF t])
      then show t ∈ {t ∈ topspace X. f t = f x} using fx t by auto
    qed
    have cD: connectedin (subtopology X {t ∈ topspace X. f t = f x}) ?D
      using pre[OF C(1)] csub by (simp add: connectedin-subtopology)
    have xd: x ∈ ?D z ∈ ?D
      using xz ab by auto
    have ?D ⊆ ?q x
      using connected-component-of-maximal[OF cD xd(1)]
      by (simp only: fibre-component-def)
    then have z ∈ ?q x using xd(2) by blast
    then have ?q z = ?q x by (rule fibre-component-eq)
    then show a = b using xz by auto
  qed
qed
```

theorem *monotone-light-factorization*:
assumes *compact-space X Hausdorff-space X*
continuous-map X Y f Hausdorff-space Y
obtains *l* **where**
compact-space (*component-quotient X f*)
Hausdorff-space (*component-quotient X f*)
quotient-map X (*component-quotient X f*) (*fibre-component X f*)
monotone-map X (*component-quotient X f*) (*fibre-component X f*)
continuous-map (*component-quotient X f*) *Y l*
light-map (*component-quotient X f*) *Y l*
l ‘ *topspace* (*component-quotient X f*) = *f* ‘ *topspace X*
⋀*x*. *x* ∈ *topspace X* ⟹ *l* (*fibre-component X f x*) = *f x*
proof −
have *eq*: *f x* = *f z* **if** *x* ∈ *topspace X z* ∈ *topspace X*
fibre-component X f x = *fibre-component X f z* **for** *x z*
using *fibre-component-self*[*OF that*(*1*), *of f*]
fibre-component-subset[*of X f z*] *that* **by** *auto*
obtain *l* **where** *l*: *continuous-map* (*component-quotient X f*) *Y l*
l ‘ *topspace* (*component-quotient X f*) = *f* ‘ *topspace X*
⋀*x*. *x* ∈ *topspace X* ⟹ *l* (*fibre-component X f x*) = *f x*
using *quotient-map-lift-exists*[*OF component-quotient-map assms*(*3*) *eq*] **by** *blast*
show *thesis*
using *that component-quotient-compact-Hausdorff*[*OF assms*]
component-quotient-map[*of X f*] *component-quotient-monotone*[*of X f*] *l*
component-quotient-light[*OF assms l*(*1*,*3*)] **by** *blast*
qed

3.2 Uniqueness

In an arbitrary factorization, connectedness and lightness force its first-map fibres to coincide with the components used in the construction.

lemma *monotone-light-fibres*:
assumes *p*: *continuous-map X Z p monotone-map X Z p*
and *k*: *light-map Z Y k*
and *eq*: ⋀*x*. *x* ∈ *topspace X* ⟹ *k* (*p x*) = *f x*
and *x*: *x* ∈ *topspace X*
shows {*z* ∈ *topspace X*. *p z* = *p x*} = *fibre-component X f x*
proof −
let *?D* = {*z* ∈ *topspace X*. *p z* = *p x*}
let *?F* = {*z* ∈ *topspace X*. *f z* = *f x*}
let *?C* = *fibre-component X f x*
have *D*: *connectedin X ?D ?D* ⊆ *?F x* ∈ *?D*
proof −
show *connectedin X ?D* **using** *p*(*2*) **unfolding** *monotone-map* **by** *auto*
show *?D* ⊆ *?F*
proof (*rule subsetI*)
fix *z* **assume** *z*: *z* ∈ *?D*
show *z* ∈ *?F* **using** *eq*[*of z*] *eq*[*OF x*] *z* **by** *auto*

```
    qed
    show x ∈ ?D using x by auto
  qed
  have cD: connectedin (subtopology X ?F) ?D
    using D by (simp add: connectedin-subtopology)
  have DC: ?D ⊆ ?C
    unfolding fibre-component-def
    by (rule connected-component-of-maximal[OF cD D(3)])
  have pc: connectedin Z (p ‘ ?C)
    using connectedin-continuous-map-image[OF p(1) fibre-component-connected]
.
  have sub: p ‘ ?C ⊆ {z ∈ topspace Z. k z = f x}
    using continuous-map-image-subset-topspace[OF p(1)]
      eq fibre-component-subset[of X f x] by auto
  have p z = p x if z ∈ ?C for z
    using k pc sub that fibre-component-self[OF x, of f]
    unfolding light-map-def by blast
  then have ?C ⊆ ?D
    using fibre-component-subset[of X f x] by blast
  with DC show ?thesis by blast
qed

theorem monotone-light-unique:
  assumes X: compact-space X Hausdorff-space X
    and f: continuous-map X Y f and Y: Hausdorff-space Y
    and l: ⋀x. x ∈ topspace X ⟹ l (fibre-component X f x) = f x
    and Z: Hausdorff-space Z
    and p: continuous-map X Z p monotone-map X Z p
      p ‘ topspace X = topspace Z
    and k: continuous-map Z Y k light-map Z Y k
    and eq: ⋀x. x ∈ topspace X ⟹ k (p x) = f x
  obtains h where
    homeomorphic-map (component-quotient X f) Z h
    ⋀x. x ∈ topspace X ⟹ h (fibre-component X f x) = p x
    ⋀A. A ∈ topspace (component-quotient X f) ⟹ k (h A) = l A
    ⋀h'. (⋀x. x ∈ topspace X ⟹ h' (fibre-component X f x) = p x)
      ⟹ ∀A∈topspace (component-quotient X f). h' A = h A
proof −
  let ?Q = component-quotient X f
  let ?q = fibre-component X f
  have fibres: ?q x = ?q z ⟷ p x = p z
    if x ∈ topspace X z ∈ topspace X for x z
  proof −
    have pf: {t ∈ topspace X. p t = p x} = ?q x
      by (rule monotone-light-fibres[OF p(1,2) k(2) eq that(1)])
    have zs: z ∈ ?q z by (rule fibre-component-self[OF that(2)])
    show ?thesis
    proof
      assume ?q x = ?q z
```

```
      then have z ∈ ?q x using zs by simp
      then have z ∈ {t ∈ topspace X. p t = p x}
        by (simp only: pf)
      then show p x = p z by auto
    next
      assume p x = p z
      then have z ∈ {t ∈ topspace X. p t = p x} using that(2) by auto
      then have z ∈ ?q x by (simp only: pf)
      then have ?q z = ?q x by (rule fibre-component-eq)
      then show ?q x = ?q z by simp
    qed
  qed
  have respects: p x = p z
    if x ∈ topspace X z ∈ topspace X ?q x = ?q z for x z
    using fibres[OF that(1,2)] that(3) by simp
  obtain h where h: continuous-map ?Q Z h h ‘ topspace ?Q = p ‘ topspace X
    ⋀x. x ∈ topspace X ⟹ h (?q x) = p x
   using quotient-map-lift-exists[OF component-quotient-map[of X f] p(1) respects]
by blast
  have inj: inj-on h (topspace ?Q)
  proof (rule inj-onI)
    fix A B assume AB: A ∈ topspace ?Q B ∈ topspace ?Q h A = h B
    obtain x where x: x ∈ topspace X ?q x = A
      using AB(1) by (auto simp: component-quotient-topspace)
    obtain z where z: z ∈ topspace X ?q z = B
      using AB(2) by (auto simp: component-quotient-topspace)
    have p x = p z using h(3)[OF x(1)] h(3)[OF z(1)] x(2) z(2) AB(3) by auto
    then have ?q x = ?q z using fibres[OF x(1) z(1)] by simp
    then show A = B using x(2) z(2) by simp
  qed
  have homeo: homeomorphic-map ?Q Z h
    using continuous-imp-homeomorphic-map[OF h(1)
       component-quotient-compact-Hausdorff(1)[OF X f Y] Z] h(2) p(3) inj by
blast
  have commute: k (h A) = l A if A: A ∈ topspace ?Q for A
  proof −
    obtain x where x: x ∈ topspace X ?q x = A
      using A by (auto simp: component-quotient-topspace)
    show ?thesis using h(3)[OF x(1)] l[OF x(1)] eq[OF x(1)] x(2) by auto
  qed
  have unique: ∀ A∈topspace ?Q. h′ A = h A
    if u: ⋀x. x ∈ topspace X ⟹ h′ (?q x) = p x for h′
  proof (intro ballI)
    fix A assume A ∈ topspace ?Q
    then obtain x where x: x ∈ topspace X ?q x = A
      by (auto simp: component-quotient-topspace)
    show h′ A = h A using h(3)[OF x(1)] u[OF x(1)] x(2) by auto
  qed
  show thesis using that homeo h(3) commute unique by blast
```

qed

end

References

[1] A. Carboni, G. Janelidze, G. M. Kelly, and R. Paré. On localization and stabilization for factorization systems. *Applied Categorical Structures*, 5:1–58, 1997. DOI: https://doi.org/10.1023/A:1008620404444.

[2] S. Eilenberg. Sur les transformations continues d'espaces métriques compacts. *Fundamenta Mathematicae*, 22:292–296, 1934. DOI: https://doi.org/10.4064/fm-22-1-292-296.

[3] N. Krofta. Topological groups. *Archive of Formal Proofs*, 2024. https://isa-afp.org/entries/Topological_Groups.html.

[4] E. Michael. Cuts. *Acta Mathematica*, 111:1–36, 1964. DOI: https://doi.org/10.1007/BF02391006.